

Why the Adversarial Review Changes Were Improvements

claude-conscript-plugin skill revisions

conmage

2026-05-09

What Changed

Three files were revised after two adversarial review agents examined the initial versions. Total: 376 lines reduced to 123 (67% cut).

File	Before	After	Cut
<code>create-study/SKILL.md</code>	150 lines	39 lines	74%
<code>examples/SKILL.md</code>	82 lines	34 lines	59%
<code>usage-guide.md</code>	168 lines	50 lines	70%

Why These Are Improvements, Not Just Cuts

1. Eliminating duplication prevents drift and confusion

The original `create-study` restated content from three existing skills: `coding` (study template, naming conventions), `conscript-coding` (form patterns, bot models), and `study-review` (the review checklist). When Claude loads the plugin, all skills are available. Restating their content in `create-study` creates two problems: (a) if the originals are updated, the copies become stale and contradictory; (b) Claude receives redundant instructions that dilute the signal. The revised version says “load these skills” and stops.

2. Removing the Phase A/B ceremony matches the existing skill style

The original author (Bogdan Popa) wrote `study-review` — a comparably complex multi-step workflow — using flat numbered sections. No “phases.” The initial `create-study` introduced Phase A / Phase B labels, creating unnecessary cognitive overhead. The adversarial reviewer flagged this as inconsistent with the plugin’s established style. The revision uses `## Initialization` and `## Create Study` — same structure, no labels.

3. Auto-continuing after initialization fixes a UX failure

The original design required two separate `/create-study` invocations: first to initialize `study-config.md`, second to actually create a study. The usability reviewer (simulating a student) identified this as the single biggest friction point: “A student naturally expects `/create-study` to actually create a study, not spend the first run on setup and then stop.” The fix: after writing `study-config.md`, continue directly into study creation.

4. Cutting the interview script lets Claude adapt

The original Phase A prescribed 4 numbered questions with sub-bullets and parenthetical examples. This over-specifies — Claude will ask better questions if given the intent (“learn the project’s domain, structure, constraints, and naming conventions”) than a rigid script. The existing `study-review` skill demonstrates this: it says “ask the user what the experimental task is” without scripting the exact phrasing.

5. The examples skill was redundant; the revision is not

The original `examples/SKILL.md` (82 lines) restated what `conscript-coding` already covers (form patterns, matchmaking patterns, etc.) in a different format. The revision keeps only what `conscript-coding` does *not* provide: (a) the pattern index mapping patterns to search terms, and (b) the instruction to read `study-config.md` for repo paths. This is 34 lines of genuinely new information.

6. The usage guide now serves its audience

The original guide included two full simulated dialogue transcripts (60+ lines). The adversarial reviewer noted: “A user who needs to read a transcript to understand `/create-study` will be confused by the real interaction too.” The revision replaces transcripts with a single example prompt and a “Key concepts” section defining terms a newcomer wouldn’t know (`conscript`, `congame`, `-with-admin`, bot models) — which the original omitted entirely.

How to Verify

The quality bar is the existing skills written by the original author. Compare line counts and density:

Skill	Lines	Invocable?
<code>upload</code> (original)	67	Yes
<code>study-review</code> (original)	183	Yes
<code>coding</code> (original)	128	No
<code>create-study</code> (revised)	39	Yes
<code>examples</code> (revised)	34	No

The revised skills sit comfortably within the range established by the originals. The initial versions were 2–3x above the upper bound.